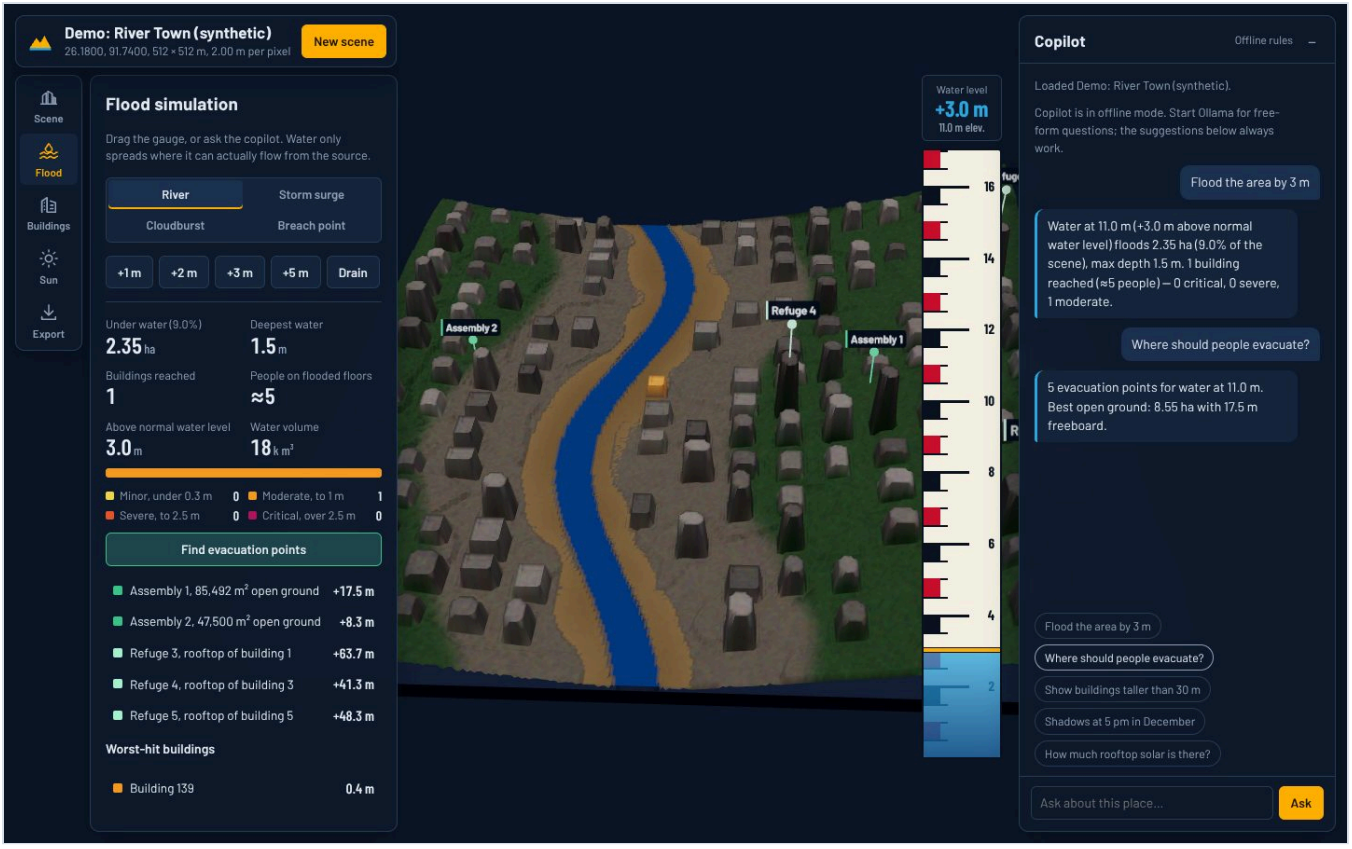
Team guide, internal. Read this before the judging round.

DepthWizard

One overhead image in, a measurable 3D twin of a neighbourhood out. Raise the river, see who floods, send people to safety, and plan rooftop solar, on a laptop with free data and free AI.



Contents

1. The project in two minutes
2. What the app does, screen by screen
3. Technology stack
4. Architecture and data flow
5. The pipeline, step by step
6. Analytics: the maths behind every number
7. The AI copilot
8. Exports: brief, AR, STL, GeoJSON
9. Engineering quality: tests, security, performance
10. Results and accuracy (memorise these)
11. Limitations, stated honestly
12. Judge and invigilator questions with answers
13. Running the demo and troubleshooting
14. Code map and glossary

1. The project in two minutes

The problem

When a river is rising, a district control room needs three answers fast: **which areas go under water, how many people are in those buildings, and where they should go.** Accurate 3D city models (from LiDAR or stereo satellite pairs) exist for only a few rich cities, cost a lot, and take months to make. Most flood-prone Indian towns have none. What every town does have is ordinary satellite imagery.

Our solution

DepthWizard builds a 3D model of a neighbourhood from **one image** and turns it into decisions:

1. An AI depth model (Depth Anything V2) estimates the 3D shape of buildings from the single image.
2. We place those buildings on **real terrain** (free SRTM elevation data) and scale their heights using **OpenStreetMap** building heights, reporting our own error honestly.
3. On that 3D model we run a **flood simulation**, count **affected buildings and people**, find **evacuation points**, show **landslide-prone slopes**, and estimate **rooftop solar**.
4. Users can ask in plain English through a **copilot** running on a free local LLM.
5. Results go out as a **printable impact brief**, an **AR model on a phone**, a **3D-printable file** and **GIS data**.

Two themes, one engine

Disaster response	Smart city / planning
Flood simulation (river, storm surge, cloudburst, embankment breach), impact report, evacuation points, landslide slopes, printable brief	Building inventory (height, floors, footprint, volume, occupants), shadows at any date and time, shadow-aware rooftop solar per building, GeoJSON for GIS

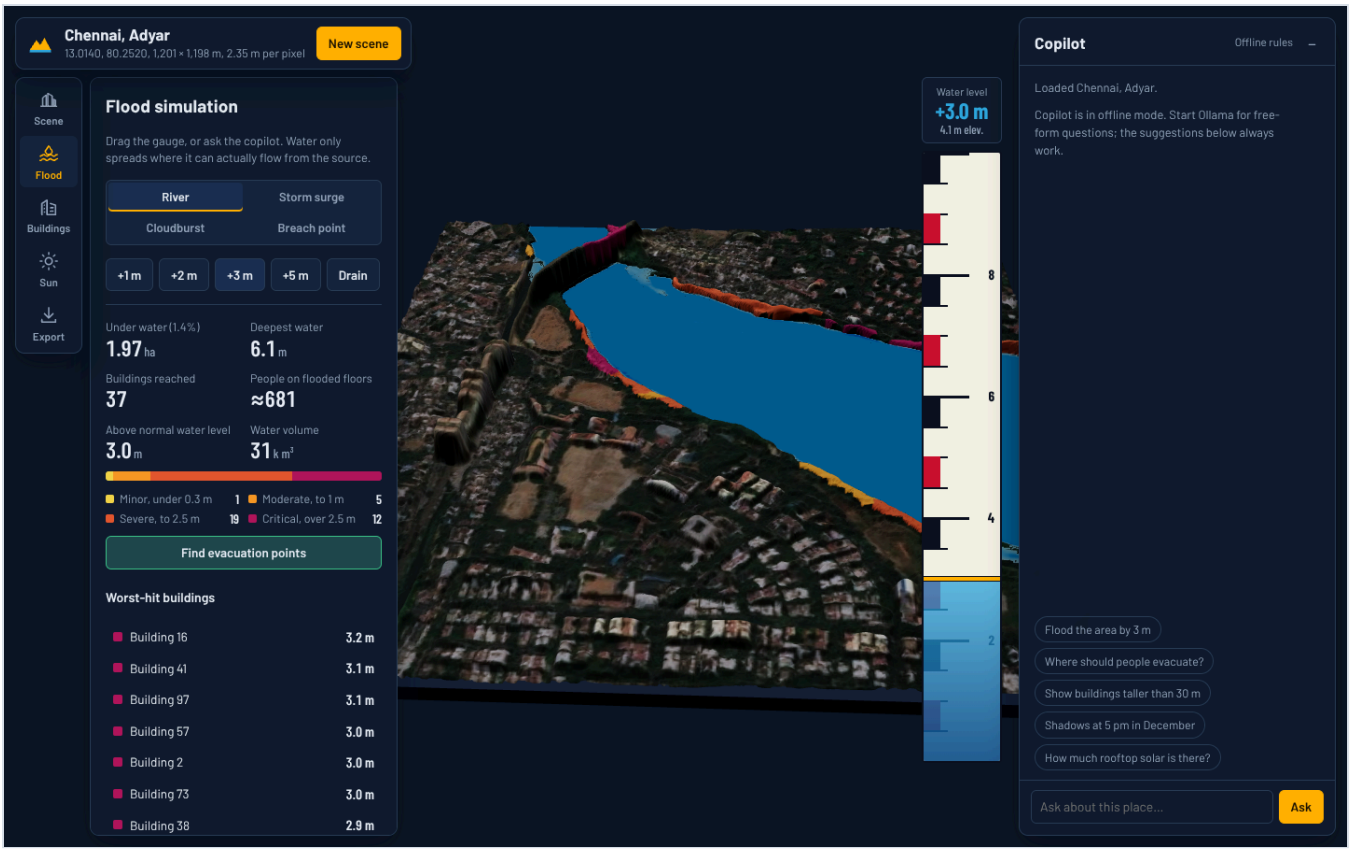
What makes it different (our "USP" lines)

- **Single image, no LiDAR, no stereo.** Works for any town that has satellite imagery.
- **Everything is free.** No API keys and no paid services: Esri imagery, AWS terrain, OpenStreetMap, open-source model, local LLM.
- **Physics, not thresholds.** Water spreads only where it can actually flow from the source.
- **Honest accuracy.** We validate on buildings we did *not* fit to, and show the error in the app.
- **The AI can't make up numbers.** The LLM only chooses actions; Python computes every figure.
- **Runs on a laptop**: CPU works, and a 6 GB GPU is fast.

> **One-line pitch to memorise**
> "One satellite image in, a measurable 3D twin out. Raise the river, see who floods, and send people to safety in under a minute, with free data and free AI on a laptop."

2. What the app does, screen by screen

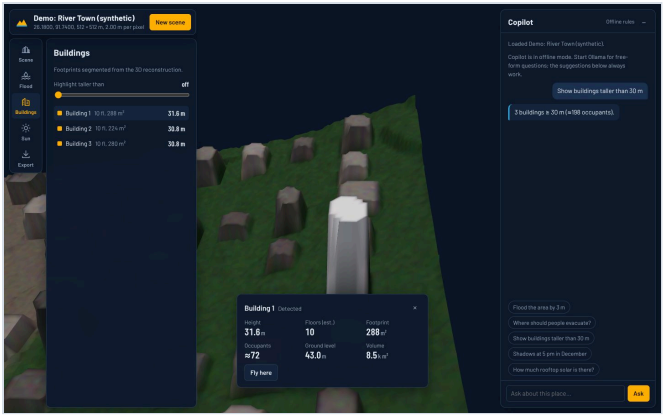
The app is a single web page served by our Python server at `http://localhost:8000`. The 3D view fills the screen. On the left are the **tool rail** and a **drawer**; on the right is the **copilot**.



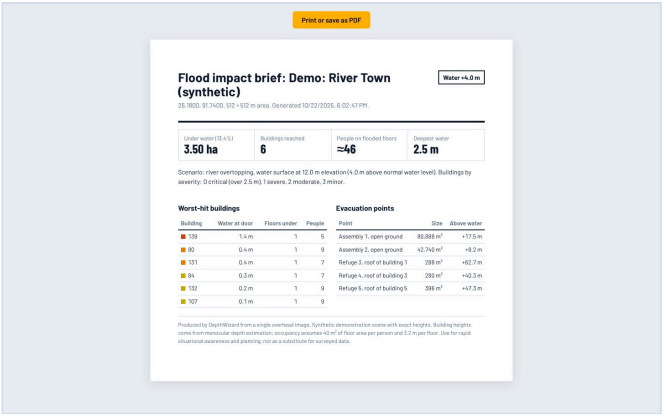
Chennai, Adyar river, with water raised 3 m. The blue river is permanent water; muddy brown is new floodwater; buildings are coloured by severity; the river staff gauge is on the right.

Part	What it does
New scene	**Pick on map**: search a place or click a preset, size the square (500 m to 2 km), then Build. **Upload image**: a drone photo, satellite screenshot or GeoTIFF, with optional pixel size and lat/lon. **Recent scenes**: reopen anything already built.
Scene tab	Summary stats (buildings, tallest, occupants, footprint, ground range, steep terrain %). **Height scale card**: how heights were calibrated and the hold-out error. Display controls: vertical exaggeration, building-edge sharpness, Photo / Heights / Slope / Mesh colouring, turntable, WASD flythrough.
Flood tab	Water source: **River**, **Storm surge** (enters from all edges), **Cloudburst** (every hollow fills), **Breach point** (click the bank). Quick buttons +1/+2/+3/+5 m and Drain. The **river staff gauge** on the right is dragged to set the level. The impact panel shows flooded area, deepest water, buildings reached, people on flooded floors, water volume, a severity bar, the worst-hit buildings and **Find evacuation points**.
Buildings tab	"Highlight taller than" slider and a list sorted by height. Clicking a building flies to it and opens a **card**: height, floors, footprint, occupants, ground level, water at the door or solar. **Match this height** recalibrates the whole scene if you know the real height of one building.

Sun tab	Month and local time put the sun in the correct position for that place, with real shadows. **Play the day** animates it. **Estimate rooftop solar** gives MWh/year, households powered, kWp capacity and the best roofs.
Export tab	A QR code to open the model on a phone (AR), .glb (AR/Blender), .stl (3D print), .geojson (GIS), screenshot, and the printable impact brief.
Copilot	Chat in plain English, e.g. "flood by 3 m and show evacuation points". It runs the actions and replies with computed numbers.
Hover readout	Under the cursor: elevation, height above ground, slope, water depth, building id.



Building card and the copilot highlighting buildings over 30 m.



The printable impact brief.

Presets (all checked against real imagery)

Srinagar (Jhelum, 2014 floods), Guwahati (Brahmaputra), Patna (Ganga), Chennai (Adyar, 2015 floods), Mumbai (Mithi river), Kochi (backwaters), Shimla (landslide slopes), Venice, Lower Manhattan. They're stored in `web/presets.json`.

The **offline demo, "River Town"**, is a synthetic town we generate in code with exact known heights. It needs no internet and no model, and it's our backup if the venue network fails.

3. Technology stack

Layer	Technology	Why we chose it
Depth AI	**Depth Anything V2** (Small by default, Base on GPU) via Hugging Face `transformers` and PyTorch	State-of-the-art open monocular depth model, sharper edges than MiDaS. Small is about 25M parameters and runs on CPU in about 10–20 s per image. Small is Apache-2.0; Base is CC-BY-NC (fine for a hackathon).
Backend	**Python 3.11, FastAPI, Uvicorn**	Fast to write, async web framework, automatic API docs at `/docs`
Numerical	**NumPy, SciPy** (`scipy.ndimage`), Pillow	Morphology, labelling, watershed, distance transforms: everything is grid maths
Geo	**rasterio, pyproj**	Reading GeoTIFFs and converting coordinate systems
Imagery	**Esri World Imagery** tiles	Free high-resolution satellite base map (attribution required)
Terrain	**AWS Terrain Tiles** (terrarium format; SRTM, NED and others)	Free global elevation, no API key
Buildings	**OpenStreetMap** via the Overpass API	Free footprints, plus height tags used as ground truth
Geocoding	Nominatim (OSM)	Free place search
3D frontend	**Three.js** (WebGL), plain ES modules, no build step	Runs in any browser; nothing to install for viewers
Map picker	**Leaflet**	Lightweight map library
AR	**Google model-viewer** + our GLB writer	One tag gives AR on Android (Scene Viewer / WebXR) and iOS (Quick Look)
Copilot LLM	**Ollama** running `qwen2.5:3b` locally, or any OpenAI-compatible API (Groq free tier)	Free and private; about 2 GB, so it fits next to the depth model on a 6 GB GPU
Testing	**pytest**, Node (for JS parity), GitHub Actions CI	37 automated tests run on every push
Deploy	Local laptop (setup/start scripts), Dockerfile available	Demo runs offline from pre-built scenes

> **Common trap.** We do *not* train any model. We use a pretrained depth model and add our own geospatial pipeline and analytics on top. That combination is the contribution. If asked "what did you train?", answer: "Nothing. The novelty is turning a generic depth model into a calibrated, validated, decision-ready 3D twin."

4. Architecture and data flow

```
BROWSER  index.html + js/main.js (controller)   viewer.js (Three.js)   flood.js   sun.js
         newscene.js (map/upload dialog)   ar.html (phone AR)   brief.html (print report)
                        |  REST/JSON + raw float32 grids
SERVER   api/main.py  (FastAPI)
           |-- src/jobs.py       single background worker queue, status.json per job
           |-- src/pipeline.py   runners: map area | upload | GeoTIFF | synthetic demo
           |-- src/tiles.py      Esri imagery + AWS terrain tiles (Web Mercator)
           |-- src/osm.py        OSM footprints, height tags, hold-out calibration
           |-- src/scene.py      depth → metric 3D surface, water detection, assets
           |-- src/analytics.py  ground, buildings, flood, evacuation, slope, sun, solar
           |-- src/copilot.py    LLM / rules → actions → analytics → reply
           '-- src/exporters.py  GLB (AR) and STL (3D print) writers
```

What happens when you click "Build 3D twin"

1. The browser sends the map square (west, south, east, north) to `POST /api/scene/map` and immediately gets a **job id**.
2. The job goes into a **single-worker queue**, because there's one GPU and two jobs at once would fight over it. The browser polls `GET /api/jobs/{id}` every 0.6 s for the progress bar.
3. The worker downloads imagery and terrain, runs the depth model, calibrates against OSM (fetched **in parallel** on another thread), builds the 3D surface, extracts buildings and water, and writes files into `data/web_jobs/<job_id>/`.
4. The browser loads `/api/result/{id}`, then downloads the grids as raw binary and builds the mesh.
5. From then on, analytics requests (flood, evacuation, solar, copilot) are quick synchronous calls on the stored grids, taking 20–30 ms.

Files stored per scene

File	Content
`rgb.png, rel.npy, dem.npy, source.json`	Inputs at processing resolution (≤ 1024 px). Kept so **rescaling never re-runs the AI model**.
`scale.json`	Height scale, terrain relief, pixel size, how it was calibrated, OSM validation numbers
`heightmap.bin`	The 3D surface (DSM) in metres, float32, 512 px grid
`heightmap_sharp.bin`	The same with sharpened walls, for display only
`ground.bin`	Estimated bare earth (the surface with buildings removed)
`labels.bin`	Building id per pixel (int32)
`water.bin`	Permanent water mask (rivers, lakes)
`buildings.json, analysis.json, manifest.json`	Per-building stats, scene summary, metadata

All grids are row-major with the north row first. Sending raw float32 instead of PNG avoids 8-bit "staircase" terracing and gives exact metres in the browser.

5. The pipeline, step by step

5.1 Satellite imagery (`src/tiles.py`)

- Web maps are cut into 256×256 **tiles** in the **Web Mercator** projection (EPSG:3857). At zoom *z* the world is $256 \cdot 2^z$ pixels wide.
- Ground size of one pixel: **GSD = 40,075,016 m × cos(latitude) / (256 × 2^z)**. At zoom 17 near Chennai that's about 1.2 m per pixel.
- We fetch **one zoom level finer** than needed and shrink it to ≤ 1024 px with Lanczos filtering, which gives a sharper result than fetching at the target zoom.
- Tiles are downloaded in parallel (8 threads) and cached on disk.

5.2 Terrain (DEM)

- AWS Terrain Tiles store height in the colour channels ("terrarium" encoding): **elevation = R×256 + G + B/256 − 32768** metres.
- The underlying data is mostly **SRTM** (NASA's Shuttle Radar Topography Mission, 2000), at 30 m resolution with a vertical error of several metres.
- We apply a **median filter about 45 m wide** to remove SRTM speckle and void pits, then a light blur.
- DEM vs DSM: a **DEM** (Digital Elevation Model) is bare earth; a **DSM** (Digital Surface Model) includes buildings and trees. We build a DSM.

5.3 Depth estimation (`src/depth_extract.py`)

- Depth Anything V2 outputs **relative inverse depth** (disparity): bigger means closer to the camera. Looking straight down, closer means **higher**, so disparity ranks heights.
- The output has no units (scale-invariant). We normalise it to 0–1 using the **2nd and 98th percentiles**, so a single shiny roof can't squash everything else (robust normalisation).
- Images are capped at 1024 px; the model internally works at 518 px.

5.4 Separating buildings from fake tilt (`scene.split_structures`)

The model was trained on ground-level photos, so on overhead images it **invents a perspective tilt**: it thinks the top of the image is farther away. On the raw output, floods would always start at the top edge. Our fix:

- **Morphological grey opening** with a 90 m window: erosion (local minimum) then dilation (local maximum). It removes anything narrower than the window (buildings) and keeps the slow-changing part (tilt and terrain). Call this `ground_rel`.
- **structures = rel − ground_rel**, divided by its 99.5th percentile, so the tallest structures are about 1.
- The low-frequency part (with the fake tilt) is **thrown away** when a real DEM is available.

5.5 Building the metric 3D surface (`scene.compose_dsm`)

```
DSM (metres) = smoothed DEM  +  structures × height_ref_m
             (map/GeoTIFF scenes)

For uploads with no location:  terrain = ground_rel stretched to terrain_relief_m (default 8 m)
Water pixels are set to one flat level (20th percentile of the terrain under them).
```

`height_ref_m` is the height in metres of the tallest structures. It comes from OSM calibration, defaults to 35 m, or is set by the user.

5.6 Height calibration against OpenStreetMap (`src/osm.py`)

1. Download all building outlines in the area from Overpass. The query starts at the same time as the depth model, with a 60 s budget, and tries three mirrors.
2. Many OSM buildings have a `height` tag (e.g. "45", "100 ft") or `building:levels` (floors × 3.2 m). Untagged footprints are still kept for building outlines.
3. Draw (rasterise) each footprint onto our image grid, **erode 2 pixels** to avoid blurry edges, and take the **median** structure value inside.
4. **Hold-out validation:** buildings with even index are used to fit *k = median(real height / structure value)*; buildings with odd index are used **only** to measure error (MAE, RMSE, bias, median % error, Pearson r).
5. The final scale uses all buildings. Fewer than 4 usable buildings, or OSM failing, falls back to the assumed scale, labelled **"Assumed scale"** in the app.

> **Why the hold-out matters.** The original version validated against the same SRTM data it was calibrated on. That's **circular validation**: the error looks small by construction. Testing on buildings never used in fitting gives an honest, independent number. Judges respect this.

5.7 Detecting permanent water (`scene.detect_water`)

SRTM is at its **noisiest over water**: in Chennai the river surface varied 0–8 m with pits to −7 m. Floods would then start in random pits instead of the river. Satellite base maps have no infrared band (no NDWI), so we use these cues:

1. **Flat** in the depth structure map (water has no buildings).
2. **Smooth texture** (low local standard deviation of brightness).
3. **Not red-dominant**, because water is blue, green or grey.
4. Either **dark** (clear water) or **glassy-smooth but bright** (silty rivers like the Brahmaputra, which were brighter than the city).
5. **Large**: at least 5,000 m^2.
6. **Low in the terrain**: median DEM ≤ 40th percentile + 1.5 m. This rejected Manhattan's shaded City Hall Park.
7. Narrow bodies must **not be lined by buildings**. This rejected shadowed street canyons.

Holes smaller than 5,000 m^2 are filled (boats, glints), but **big islands stay dry**. The detected water is flattened in the DSM and becomes the **default flood source**.

5.8 Bare earth and buildings (`src/analytics.py`)

- **Ground estimate, pass 1:** grey opening of the DSM with an 80 m window, padded at the borders first (a bug we found: without padding the ground sank along the edges), then Gaussian smoothing.
- **Pass 2 (refine):** mark everything more than 2.5 m above pass 1 as "elevated", then re-interpolate the ground using only the other pixels (normalised convolution). This fixes hillsides that the opening shaves off.
- **nDSM** (normalised DSM) = DSM − ground = height above ground.
- **Buildings when OSM has ≥ 15 footprints:** use the OSM outlines and measure height from our nDSM. Each building also stores its OSM reference height.
- **Otherwise, segmentation:** nDSM > 2.5 m, then cleanup; reject blobs without "walls" (median edge slope ≤ 0.35); then **marker-controlled watershed** seeded at roof peaks at least 14 m apart, to split neighbouring roofs the depth model blurred together.

- **Per building:** height = **90th percentile** of nDSM (robust to noisy pixels), floors = round(height / 3.2 m), footprint, volume, base elevation, **occupants = footprint × floors / 40 m²**.

6. Analytics: the maths behind every number

6.1 Flood simulation (`flood_mask` , browser `flood.js`)

A naive approach floods every pixel below the water level, so a basin behind a ridge or levee would flood even though water can't reach it. We do a **connected flood fill**:

1. Candidates = pixels with surface below the water level.
2. Label connected regions using **4-connectivity** (no leaking through diagonal gaps).
3. Keep only regions that touch the **source**:
   - **River** (default): detected permanent water, or else the lowest 0.5% of pixels.
   - **Storm surge**: any edge of the scene.
   - **Cloudburst**: no connectivity rule; every hollow fills (ponding).
   - **Breach point**: the pixel the user clicked.

The browser re-implements the same fill in JavaScript (about 4 ms on 512×512), so water follows the slider at 60 fps. An automated test checks that **JS and Python give identical cells** for 24 cases.

Gauge zero = normal water level, i.e. the median surface of the detected river; without water, the lowest ground. "+3 m" means 3 m above normal river level.

6.2 Impact report (`flood_report`)

Number	How it's computed
Flooded area	Wet pixels × pixel area, **excluding the permanent river** itself
Depth	Water level − surface, per wet pixel; mean, max and volume (Σ depth × area)
Buildings reached	Water touches a 2-pixel ring around the footprint **and** level − building base > 0.05 m
Severity	Water at the door: **minor < 0.3 m, moderate < 1 m, severe < 2.5 m, critical ≥ 2.5 m**
Floors underwater	ceil(depth / 3.2 m), capped at the building's floors
People affected	occupants × floors underwater / total floors (only people on flooded floors)

6.3 Evacuation points (`safe_zones`)

- **Assembly points:** dry, open (non-building) ground, shrunk 2 pixels from the water's edge, in patches of at least **400 m²** and at least **0.5 m above water** (freeboard). Ranked by size; up to 5.
- **Vertical refuges:** up to 3 buildings whose roof is more than 6.4 m (two floors) above the water, ranked by footprint × floors.
- Each marker is snapped onto a real pixel of its zone, because the centroid of an L-shaped area can fall outside it.

6.4 Landslide slopes

Slope = arctan(|gradient|) computed on the **bare-earth ground** (so building walls don't count). Classes: **< 15° stable, 15–30° caution, > 30° hazard**. The Slope view colours the surface the same way.

6.5 Sun position

The **NOAA solar position equations** (equation of time, declination, hour angle) give azimuth and elevation for any lat/lon and time, accurate to well under 1°. The same formula is implemented in both Python and JS and tested to match to 10^{-6}°. Three.js then casts real shadows from that direction.

6.6 Rooftop solar (`solar_potential`)

1. Sample the year: the 21st of Jan, Mar, May, Jul, Sep and Nov, hourly 06:00–18:00, giving about 80 sun positions.
2. For each sun position, **ray-march shadows**: a pixel is shaded if any pixel towards the sun rises above the sun's ray.
3. Clear-sky sunlight (Meinel air-mass model): DNI = $1353 \times 0.7^{(AM^{0.678})}$, AM = 1/sin(elevation), plus 10% diffuse.
4. Sum over the year, then × 0.75 for clouds and haze → kWh/m^2/year per pixel.
5. Per roof: kWh/yr = roof area × **60% usable × 20% panel efficiency × 80% performance ratio** × sunlight. Households = total ÷ 1,200 kWh/yr.

The demo town gives about 1,490 kWh/m^2/yr, a plausible figure for India. The calculation takes under 1 s and is cached per scene.

7. The AI copilot (`src/copilot.py`)

Design rule: the LLM chooses, Python calculates

Small free models are good at understanding intent and bad at arithmetic. So the LLM's only job is to turn the sentence into a list of **actions** in JSON. Python then validates the actions, runs the real analytics and writes the reply from the computed numbers. Nothing in a reply can be invented.

```
User: "storm surge of 2 m and where should people go?"
LLM → {"actions":[{"type":"flood","rise_m":2,"source":"edges"},{"type":"safe_zones"}]}
Python → flood_report(...) + safe_zones(...) → "Water at 9.1 m floods 2.4 ha ... 5 evacuation
points ..."
Browser → animates water, places markers, opens the Flood tab
```

Actions available

`flood` (rise_m or level_m, source), `flood_clear`, `safe_zones`, `highlight_buildings` (min/max height, top N), `fly_to`, `sun` (hour, month), `solar`, `view` (rgb/slope/heights/wire), `report`.

Backends

- **Default: Ollama** on `localhost:11434`, model `qwen2.5:3b`. Temperature 0 and JSON-only output. Unknown action types are dropped (tested with a fake server that returns a "delete_everything" action).
- **Any OpenAI-compatible API**, e.g. Groq's free tier, via the `DW_LLM_BASE_URL`, `DW_LLM_API_KEY` and `DW_LLM_MODEL` environment variables.
- **Offline fallback:** a rule parser (regular expressions) that understands floods, sources, heights, times and months, solar, evacuation and views. It works without any LLM, and the engine label shows "Offline rules".

8. Exports

Export	How
Impact brief (`brief.html`)	A printable A4 page: snapshot, KPIs, scenario sentence, worst-hit buildings, evacuation points, accuracy statement and assumptions. Print → Save as PDF.
GLB (glTF 2.0 binary)	Our own writer (no extra library): a 256×256 mesh, UVs, embedded JPEG texture, scaled to a 60 cm "tabletop" model. `ar.html` shows it with model-viewer; the QR code points to the laptop's LAN address.
STL	Watertight solid in mm (top surface, four walls, base), so slicers can print it. A test checks every edge is shared exactly twice and the volume is positive.
GeoJSON	Building boxes with height, floors and occupants, in lon/lat (converted via Mercator), for QGIS or ArcGIS.

AR note: iPhone Quick Look works over plain Wi-Fi HTTP. Android WebXR/Scene Viewer requires HTTPS; use a Cloudflare tunnel if needed.

9. Engineering quality

Tests (37, all passing, run in GitHub Actions on every push)

- **Analytics vs ground truth** on the synthetic River Town: ground error, building count and height, flood monotonic and connected, basin behind a ridge stays dry, evacuation points are dry, sun position against reference values, shadows on the correct side, shaded roof gets less solar.
- **Water:** detects the synthetic river; the flood source ignores isolated DEM pits.
- **OSM:** height-tag parsing ("100 ft", levels); calibration recovers a known scale within 10%.
- **Exports:** valid GLB structure, watertight and outward-facing STL.
- **Copilot:** 8 phrasings parsed correctly, replies grounded in numbers, mock LLM server used and validated.
- **API:** health, results, flood, evacuation, exports; **security** (path traversal rejected, bad ids rejected, upload filename sanitised).
- **JS/Python parity:** runs the browser code in Node and compares it with Python cell for cell.

Security measures

- Upload filenames are sanitised. The old version saved files under the browser-sent name, so `../../` could write outside the folder (a path traversal bug we fixed).
- Upload size limit (60 MB), extension whitelist, job-id regex, asset-name whitelist.
- The server binds to the LAN so phones can connect. Set `DW_HOST=127.0.0.1` to keep it laptop-only.

Robustness

- Background job queue with on-disk status; jobs interrupted by a restart are marked instead of hanging forever.
- OSM mirrors are flaky, so we try 3 mirrors, cache responses, run the fetch in parallel, cap it at 60 s and fall back gracefully.
- Tiles are cached, so a pre-built scene works with no internet.
- PyTorch is imported lazily, so analytics and tests run without it.

10. Results and accuracy (memorise these)

Test	Result
Synthetic town: buildings found	**139 / 139**, no merges
Synthetic town: building height error	**0.99 m** mean absolute
Synthetic town: bare-earth error	**0.05 m** mean absolute
Lower Manhattan vs held-out OSM (107 fit, 53 tested)	Mean error **±33.5 m**, median 50%, r = 0.49 (skyscrapers, the worst case)
Chennai, Adyar (1.2 km)	River = 26% of scene; **+3 m reaches 36 buildings, about 660 people**
Guwahati, Brahmaputra	53% water detected, islands kept dry
Manhattan water	0%: shaded streets and park correctly rejected
Speed (CPU)	Whole scene about **47–60 s**; depth 8–20 s; flood report **23 ms**; evacuation 30 ms; solar 0.9 s; browser flood fill 3.7 ms

How to say the accuracy honestly

"Monocular depth ranks building heights well but absolute heights carry about ±50% error. That's why we calibrate against OpenStreetMap and show the error on screen. For floods the critical inputs are ground level and connectivity, which come from real terrain data, not the AI."

11. Limitations, stated honestly

Say these before the judges find them. Admitting a limitation with a fix plan scores better than being caught.

Limitation	Why	Fix / next step
Absolute heights about ±50%	Depth model trained on ground photos, no metric scale	Fine-tune on aerial LiDAR datasets (ISPRS Potsdam, DFC2019); stereo where available
Trees sometimes counted as buildings	Canopy is elevated with soft edges; satellite shadows are green-tinted, so a greenness test failed	Land-cover segmentation (e.g. SegFormer), NDVI from multispectral imagery; OSM footprints already solve it where mapped
Flyovers and bridges read as buildings	Same height signature	Shape filter or OSM `bridge=yes`
Static flood (no time, no flow speed)	Designed for minute-level triage	Couple to a 2D shallow-water solver (LISFLOOD-FP style) using our DSM and water mask
Terrain is 30 m SRTM	Best free global DEM	National DEMs such as ISRO's CartoDEM
Water detection is heuristic	No infrared in web imagery	OSM water polygons, Sentinel-2 NDWI
Occupancy is an estimate	40 m^2/person, uniform	Census / WorldPop population rasters
Solar is clear-sky × 0.75	No weather data	NASA POWER / PVGIS irradiance
Off-nadir imagery	Building lean breaks the height assumption	RPC-based orthorectification
Area up to about 4 km	Browser mesh 512×512, tile budget	Tiled level-of-detail streaming
Building a scene needs internet	Tiles and OSM are online	Pre-build with `dw.py presets`; cached afterwards

12. Judge and invigilator questions, with answers

Concept and impact

Q. What problem are you solving and for whom?
District disaster management and municipal planners in towns without 3D city data. When a river rises they need to know which buildings flood, how many people are affected and where to evacuate, in minutes rather than months.

Q. How is this different from Google Earth 3D or Bhuvan?
Google's 3D exists only for some cities and you can't run analysis on it. Bhuvan provides data and maps but no instant single-image 3D reconstruction with flood impact counts. We create a 3D model for *any* place from one image and run flood, evacuation and solar analysis on it, with exports to GIS.

Q. Who would actually use it?
SDMA/DDMA control rooms for flood triage, municipal corporations for planning and solar programmes, NGOs in relief work, and students and researchers. Running on a laptop with free data matters for small towns.

Q. What does it cost to run?
Nothing. Imagery, terrain and OSM are free, the depth model is open source, and the LLM runs locally. It needs only a laptop; a 6 GB GPU makes it faster.

AI and model

Q. Which model do you use and why?
Depth Anything V2 (Small on CPU, Base on GPU). It's a state-of-the-art open monocular depth model trained on very large synthetic and real datasets, with sharper building edges than MiDaS. Small is about 25M parameters.

Q. Did you train or fine-tune it?
No. We use it pretrained. Our contribution is the pipeline: removing its perspective tilt, anchoring it to real terrain, calibrating against OSM with hold-out validation, water detection and the analytics. Fine-tuning on aerial LiDAR data is our next step.

Q. How can one image give height? Isn't depth from one image impossible?
Geometrically it's ambiguous, but deep networks learn cues such as shadows, occlusion, texture and familiar shapes. The output is *relative* depth: correct ordering, unknown scale. That's exactly why we add a scale from OSM and real terrain from SRTM.

Q. What is disparity and why does it map to height?
Disparity is inverse depth, 1 / distance. Looking straight down, a roof is closer to the camera than the street, so higher disparity means higher elevation. This holds for near-vertical (nadir) views.

Q. The model was trained on street photos. Why does it work on satellite images?
It generalises well at outlining buildings, but it invents a perspective tilt. We remove that with a morphological opening (a 90 m window) and keep only the sharp structures, then use real terrain instead of the model's guess.

Q. How accurate are your heights?
On synthetic ground truth, about 1 m. On real Lower Manhattan, checked on 53 OSM buildings not used in fitting, the mean error was 33.5 m, about 50%, because skyscrapers are the hardest case. Heights are reliable for ranking and triage, not for surveying. We show this error in the app.

Q. Isn't validating on OSM risky? OSM can be wrong.

Yes, OSM tags have their own errors, but they are independent of our model and free everywhere. The key point is we test on buildings *not used for fitting*, which avoids circular validation. With LiDAR data we would validate against that.

Geospatial

Q. What is SRTM? What are DEM, DSM and nDSM?

SRTM is NASA's 2000 Shuttle Radar Topography Mission: near-global elevation at 30 m. A DEM is bare earth; a DSM includes buildings and trees; nDSM = DSM – DEM, the height above ground.

Q. Which projection and coordinate system?

Web Mercator (EPSG:3857) for tiles; metres per pixel = $40{,}075{,}016 \times \cos(\text{lat}) / (256 \times 2^{\text{zoom}})$. GeoJSON exports use WGS84 lon/lat (EPSG:4326). GeoTIFF uploads keep their own CRS, and bounds are reprojected with rasterio.

Q. What is GSD?

Ground Sample Distance: the size on the ground of one pixel. Map scenes are about 0.6–2.3 m per pixel at processing resolution; the 3D mesh is 512 px.

Q. Why not use NDWI for water?

NDWI needs a near-infrared band, and web satellite tiles are RGB only. We combine colour, texture, flatness, terrain position and shoreline context instead. With Sentinel-2 imagery, NDWI would be the upgrade.

Flood model

Q. How does your flood simulation work? Is it real hydrology?

It's a connectivity-based inundation model: a pixel floods if it is below the water level *and* connected (4-neighbour) to the water source. That's the first-order physics of where water can reach. It's not a time-dependent hydrodynamic model (no velocities or rainfall timing); that would be the next step using our DSM as input.

Q. Why not just threshold "below X metres"?

Thresholding floods basins behind ridges and embankments that water can't reach, which overestimates. Our test has a basin behind a ridge that stays dry.

Q. Where does "+3 m" start from?

From normal water level: the median surface of the detected river. Without a river, it's the lowest ground.

Q. How do you count affected people?

Occupants per building = footprint × floors ÷ 40 m^2 per person, with floors = height ÷ 3.2 m. Affected = occupants on floors below the water (ceil(depth ÷ 3.2 m) floors). It's a planning estimate and the assumptions are printed; census or WorldPop data would refine it.

Q. How are evacuation points chosen?

Dry open ground patches at least 400 m^2 and at least 0.5 m above water, ranked by size, plus up to 3 tall buildings whose roofs are more than 6.4 m above water as vertical refuges.

Q. Why is the river blue and the flood brown?

Blue is the permanent river; brown is new floodwater carrying silt. You can see at a glance what's newly flooded.

Solar and sun

Q. How do you calculate solar potential?

About 80 sun positions over a year from the NOAA equations; for each, we ray-march shadows through our 3D model, add up clear-sky sunlight (Meinel model), derate by 25% for clouds, and apply 60% usable roof, 20% panel efficiency and an 80% performance ratio. Because shadows come from our own 3D city, a tower shading a neighbour is counted.

Copilot

Q. Which LLM? Doesn't it hallucinate?

Qwen 2.5 3B running locally via Ollama (free, private), or any OpenAI-compatible API. It can't hallucinate numbers because it only outputs actions in JSON; Python validates them and computes every figure. Without an LLM, a rule-based parser handles common requests.

Q. What if the LLM returns something dangerous or invalid?

Only nine known action types are accepted; anything else is dropped. Our tests include a fake LLM returning "delete_everything", which is ignored.

Engineering

Q. Why FastAPI and Three.js, not Unity or React?

The browser means zero install for viewers, including phones. Plain ES modules mean no build step. FastAPI is fast and simple. Unity builds are heavy and need installation.

Q. How do you handle multiple users?

A single background worker queue, since there's one GPU; others wait with a queue position shown. Analytics calls are fast (20–30 ms) and run concurrently.

Q. How did you test it?

37 automated tests in CI, with ground truth from a synthetic town where we know every building height. Real-scene validation on Chennai, Guwahati and Manhattan, and a test that the browser and server flood code give identical results.

Q. What about security?

We fixed a path traversal bug in uploads, added size limits and whitelists for ids and assets. It's a demo server, so authentication would be added before any public deployment.

Q. Can it run offline?

Yes, for viewing and analysing pre-built scenes and the synthetic demo. Building a new map scene needs internet for tiles and OSM, which are cached afterwards.

Q. How long does a scene take?

About a minute on a CPU laptop: roughly 2 s tiles, 10–20 s depth, up to 60 s OSM (in parallel), under 1 s analytics. Much faster on a GPU.

Future

Q. What would you do with more time or funding?

Fine-tune on aerial LiDAR; land-cover segmentation to separate trees; CartoDEM terrain; a rainfall-runoff hydrodynamic model; WorldPop population; integration with live river-gauge feeds (e.g. CWC) so the scenario updates automatically; a mobile field mode.

13. Running the demo and troubleshooting

Setup (once)

```
cd depth-wizard3D-main            (the folder containing dw.py)
.\setup_laptop.bat                (detects an NVIDIA GPU and installs PyTorch with CUDA or CPU)
ollama pull qwen2.5:3b            (optional: free copilot; install Ollama from ollama.com first)
```

Every time

```
.\start_server.bat                → http://localhost:8000   (Ctrl+C stops it)
```

The night before

```
.venv\Scripts\python.exe dw.py presets      (pre-build all preset places; works offline later)
.venv\Scripts\python.exe dw.py list         (show built scenes)
```

3-minute demo order

1. Open **Chennai, Adyar** from Recent scenes. Say: "This is from one satellite image."
2. Orbit, then point to the height-scale card: "Calibrated against OSM; this is our honest error."
3. Flood tab: drag the gauge to +3 m and read the impact numbers aloud.
4. **Find evacuation points**.
5. Copilot: "storm surge of 2 m and where should people evacuate".
6. **Breach point**, then click the bank.
7. Export: open the impact brief; let a judge scan the QR for AR.
8. Sun tab: **Play the day**, then **Estimate rooftop solar**.

Backup if the internet or model fails: open the **River Town** demo. Every feature works on it offline.

Troubleshooting

Problem	Fix
"Python was not found"	Install Python 3.11 with "Add Python to PATH" ticked, then reopen VS Code
Double-click does nothing in VS Code	Run from the terminal: `.\start_server.bat`, in the folder that contains `dw.py`
Port 8000 busy	`set DW_PORT=8001`, start again, open localhost:8001
Copilot says "Offline rules"	Ollama not running: start Ollama and check `ollama list` shows qwen2.5:3b
"Assumed scale" on a map scene	OSM servers timed out. Rebuild later, or select a building of known height and use **Match this height**
Phone can't open the QR link	Same Wi-Fi; allow Python through Windows Firewall (private networks)
GPU not used	`nvidia-smi` must work; update drivers and rerun setup

Environment variables

Variable	Meaning (default)

`DW_MODEL_SIZE`	small | base | large (small; start scripts use base on GPU)
`DW_DEVICE`	auto | cuda | cpu (auto)
`DW_PORT`, `DW_HOST`	Server port and bind address (8000, 0.0.0.0)
`DW_LLM_BASE_URL`, `DW_LLM_MODEL`, `DW_LLM_API_KEY`	Copilot backend (Ollama localhost:11434/v1, qwen2.5:3b, none)
`DW_LLM=0`, `DW_OSM=0`	Disable the LLM / OSM calls
`DW_PUBLIC_URL`	HTTPS tunnel URL for the AR QR code

14. Code map and glossary

File	Responsibility
`api/main.py`	FastAPI routes: create scenes, job status, results, assets, flood, evacuation, solar, copilot, scale, exports, health
`src/pipeline.py`	Scene runners for map, upload, GeoTIFF and demo; the shared depth model; parallel OSM fetch
`src/tiles.py`	Mercator maths, zoom choice, tile mosaics, terrarium decoding
`src/osm.py`	Overpass query, height parsing, rasterising, hold-out calibration
`src/scene.py`	Tilt/structure split, water detection, DSM composition, asset build
`src/analytics.py`	Ground, buildings, flood, report, evacuation, slope, sun, shadows, solar
`src/copilot.py`	Rule parser, LLM parser, action execution, replies
`src/exporters.py`	GLB and STL writers
`src/jobs.py`	Background queue, status files
`src/synthetic.py`	River Town generator (tests and offline demo)
`src/mesh_export.py`, `depth_extract.py`, `calibrate.py`, `georef.py`, `validate.py`	Original research modules: web heightmaps, model wrapper, RANSAC calibration, GeoTIFF IO, metrics
`web/js/viewer.js`	Three.js scene: mesh, water, overlays, markers, sun, picking, camera
`web/js/main.js`	UI controller: tabs, gauge, panels, copilot chat, exports
`web/js/flood.js, sun.js`	Browser copies of the flood fill and sun maths (parity-tested)
`web/ar.html, brief.html`	Phone AR page; printable impact brief
`dw.py`	Command-line scene builder
`tests/`	37 pytest tests

Glossary

Monocular depth	Estimating depth from a single image
Disparity	Inverse depth; larger means closer
DEM / DSM / nDSM	Bare earth / surface including buildings / height above ground
SRTM	NASA 30 m global elevation dataset (2000)
GSD	Ground size of one pixel, in metres
Web Mercator	Projection used by web maps (EPSG:3857)
Morphological opening	Erosion then dilation; removes features narrower than the window
Watershed	Segmentation that "floods" an image from seed points to split touching objects
Normalised convolution	Blur that ignores masked pixels; used to interpolate ground under buildings
4-connectivity	Pixels connect only up, down, left and right, not diagonally

Freeboard	Height above the water surface
Hold-out validation	Testing on data not used for fitting
Circular validation	Testing on the same data used to fit, which looks good by construction
MAE / RMSE / r	Mean absolute error / root-mean-square error / Pearson correlation
NDWI / NDVI	Water / vegetation indices that need an infrared band
GLB / STL / GeoJSON	3D model for AR / 3D-print mesh / GIS vector data
Ray marching	Stepping along a ray through the height grid to test for shadow
kWp	Kilowatt-peak: panel capacity under standard test conditions
Ollama	Tool for running LLMs locally

Data credits: Esri World Imagery (Esri, Maxar, Earthstar Geographics); AWS Terrain Tiles (SRTM and others); © OpenStreetMap contributors (ODbL); Depth Anything V2; Three.js, Leaflet, Google model-viewer.